

WHAT IS DATA, DATABASE?

- **Data** is any thing that can be represented as text, numbers, or multimedia(image, sound, video etc.)

Examples: 'Technology', 243589, '2022-09-20'

- **Dataset** is a structured collection of data that contains information of an entity

Examples:

CUST_ID	CUST_NAME	AGE	EMAIL	PHONE
1011	John	31	john@gmail.com	1234567890
1022	Smith	34	smith@yahoo.com	2345678910
1023	Jessica	29	jessica@microsoft.com	6789012345
1024	Charles	27	charles@gmail.com	4561239087

EmpId, Name, Sal, Age, Deptid, LocId
123, Ramu, 20000, 26, 101, 11
456, Latha, 31000, 28, 102, 22
789, Ravi, 28000, 32, 103, 33

- **Database** is a system that allow users to store, organize and analyze data. It is an organized collection of data stored in multiple related datasets.

Example: Employee Database contains below datasets.

DEPT dept_id dept_name desc	EMPLOYEE emp_id emp_name dob salary dept_id loc_id	LOCATION loc_id loc_name desc
SAL_GRADE grade min_sal max_sal		HIERARCHY emp_id mngn id

Database is collection of related datasets to analyse the data simply. There should be a relation between the datasets in a database.

USES OF DATABASE?

- Can handle huge amounts of data
- Easy to combine different datasets
- Analyze the data for decision making
- Database is backend for many applications
- Easy to share the data
- Data security

WHAT IS RDBMS?

RDBMS – Relational Data Base Management Systems.

- A relational Database is database that stores data in a table form, containing columns and rows.
- RDBMS is a system used to store, manage, query, and retrieve data stored in a relational databases.
- The RDBMS provides an interface between users&applications and the database.

LIST OF RDBMS SYSTEMS

- Oracle
- MySQL
- Teradata
- SQL Server
- PsotgreSQL
- Vertica
- IBM DB2
- Snowflake (Cloud Database)

WHAT IS TABLE?

- **Table** is a database object that is a collection of related data entries, where data is organized in rows and columns.
- Rows are also know as records
- Columns are also known as fields

Example: Customer table

CUST_ID	CUST_NAME	AGE	EMAIL	PHONE
1011	John	31	john@gmail.com	1234567890
1022	Smith	34	smith@yahoo.com	2345678910
1023	Jessica	29	jessica@microsoft.com	6789012345
1024	Charles	27	charles@gmail.com	4561239087

Metadata: Data about data, it gives the info that what data is stored in a table.

HOW TO LOAD DATA INTO DATABASE TABLES?

- Using manual insert statements
- Importing data from files using SQL clients(Dbeaver, Squirrel, Oracle SQL dev, Teradata SQL Assistant, TOAD)
- Load using ETL tools or Scripts

Scripts means java or python(any) programm

HOW TO ACCESS THE DATA FROM DATABASE?

SQL – Structured Query Language

- We can access and manipulate the data from relational databases by using SQL.
- SQL is a programming language used to communicate with these RDBMS systems like Oracle, Teradata, MySQL, SQL Server etc.
- SQL is an ANSI/ISO standard, all databases support ANSI SQL so the syntaxes, keywords and functions are almost similar in all databases.
- We can build SQL queries and procedures to operate with the data present in Database.
- Queries can be written by using Operators, Keywords, Expressions, Functions..

DATA TYPES

- Data types specify the type of data we storing or querying.
- There are many data types to store
 - Text (characters, strings)
 - Numeric values
 - Date and Time values
 - Boolean (True or False)
 - Images, audio, video data

DATA TYPES

CHAR(size)	A FIXED length string, it can contain letters, numbers, and special characters. The size parameter specifies the column length in characters.
VARCHAR(size)	A VARIABLE length string, it can contain letters, numbers, and special characters. The size parameter specifies the maximum string length in characters. Takes less space than CHAR.
INT	It is used for the integer value. Its range is from -2147483648 to 2147483647.
INTEGER	Same as INT
BIGINT	A large integer. Its range is from -9223372036854775808 to 9223372036854775807.
FLOAT(size, d)	A floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter.
DOUBLE(size, d)	A normal-size floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter
NUMBER(size, d)	Number can be used instead of Integers and decimals
DECIMAL(size, d)	Same as Number

Activate
Go to Site

DATE	It is used to specify date format YYYY-MM-DD
TIME	It is used to specify the time format. Its format is HH:MI:SS or HH24:MI:SS
DATETIME	It is used to specify date and time combination. Its format is YYYY-MM-DD HH:MI:SS Supported range for DATETIME is '1000-01-01 00:00:00' to '9999-12-31 23:59:59'
TIMESTAMP	It is used to specify the timestamp. Supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC.
BOOLEAN	Just to indicate True or False, 0 is False, any non-zero number is True
Others	BLOB, CLOB, BINARY, VARBINARY to store multimedia information like images, sounds and videos.

OPERATORS IN SQL

SQL Arithmetic Operators:

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo

SQL Comparison Operators:

Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<> or !=	Not equal to

SQL Logical Operators:

Operator	Description
AND	TRUE if all the conditions separated by AND is TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
NOT	Displays a record if the condition(s) is NOT TRUE
ALL	TRUE if all of the subquery values meet the condition
ANY	TRUE if any of the subquery values meet the condition
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records

SQL Bit-wise Operators:

Operator	Description
&	Bit-wise AND
	Bit-wise OR
^	Bit-wise Exclusive OR
~	Bit-wise NOT

SQL Set Operators:

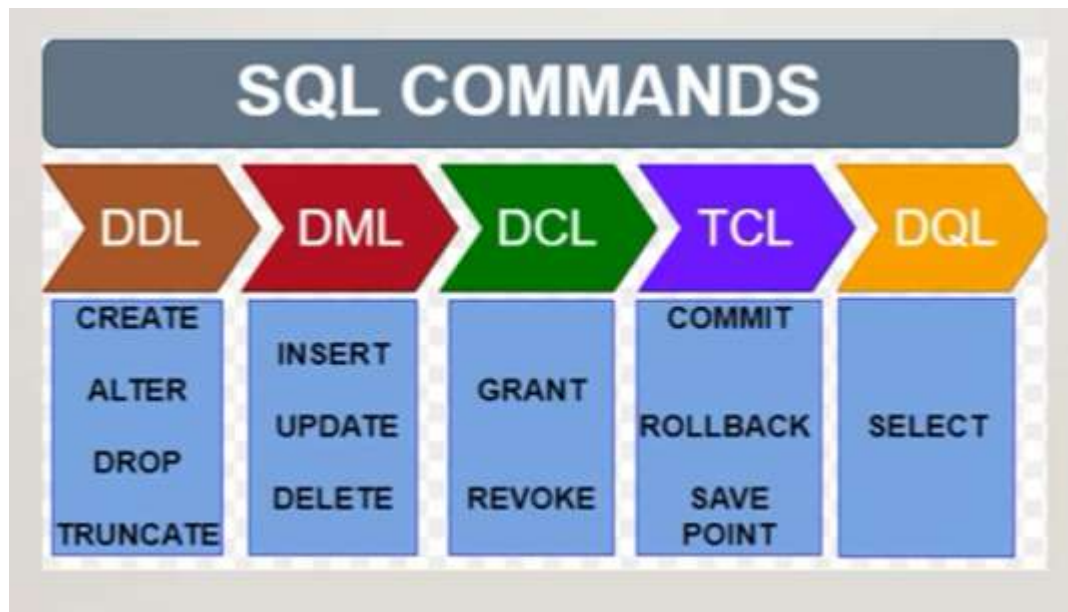
UNION
UNION ALL
INTERSECT
MINUS

Schema's are database level objects (DB lo schema vuntaay)
Tables are schema level objects (Schema lo Table vuntaay)

TYPES OF SQL STATEMENTS

There are 5 types of SQL statements.

1. DDL – Data Definition Language
2. DML – Data Manipulation Language
3. DCL – Data Control Language
4. TCL – Transaction Control Language
5. DQL – Data Query Language



DDL (DATA DEFINITION LANGUAGE)

DDL: Deals with the definition of database objects, these commands used to create, modify and remove the structure of the database objects like tables, views, functions etc.

Below are the DDL commands

- Create
- Alter
- Drop
- Truncate

CREATE

CREATE: To creates objects in the database like tables, indexes, functions, views, procedures and triggers etc.

Syntax: CREATE TABLE *TAB_NAME*

```
( Column1 DataType,  
  Column 2 DataTpe,  
  .....  
);
```

```
CREATE VIEW VIEW_NAME as SELECT ... FROM TAB_NAME;
```

ALTER

ALTER: Used to alter the structure of table like adding new columns, dropping existing column, changing the length or datatype of a column and to rename the table.

Syntax:

```
ALTER TABLE TABLE_A RENAME TO TABLE_B;  
ALTER TABLE TABLE_NAME ADD new_column datatype;  
ALTER TABLE TABLE_NAME MODIFY COLUMN column_name;  
ALTER TABLE TABLE_NAME DROP COLUMN column_name;
```

```
-- Increase the length of first_name from 20 to 25  
ALTER TABLE public.employees modify first_name VARCHAR(25);  
  
-- Drop existing column manager_id  
ALTER TABLE public.employees drop column manager_id;
```

DROP

DROP: Used to drop database objects like tables, views, functions, procedures, indexes etc.

Syntax:

```
DROP TABLE TABLE_NAME; or DROP TABLE_NAME;  
DROP VIEW VIEW_NAME;  
DROP FUNCTION FUN_NAME;  
DOPR INDEX INDEX_NAME;
```

TRUNCATE

TRUNCATE: It deletes all rows from the table and we can't rollback the deleted records.

Syntax:

```
TRUNCATE TABLE TABLE_NAME;
```

DML (DATA MANIPULATION LANGUAGE)

DML: DML statements are used to add the data, modify data and to delete the data from tables.

Below are the DML commands

- Insert
- Update
- Delete

INSERT

INSERT: To insert data into a table, by using INSERT we can

1. Insert single record manually
2. Insert multiple records manually
3. Insert bulk data from other processes
4. Insert data from one table to another table

Syntax: INSERT INTO table_name(columns)
VALUES(.....);

INSERT INTO table_a(columns) SELECT columns FROM table_b;

UPDATE

UPDATE: To modify data from the table. We use SET keyword for updating data. By using UPDATE we can

1. Update one or more columns data
2. Update one or more records based on condition
3. Update entire table

Syntax: UPDATE TABLE table_name SET column1=... , column2=...;

UPDATE TABLE table_name SET column1=...
WHERE condition;

```
UPDATE public.employees  
SET dept_id=70 where dept_id=60;
```

```
-- Increase salary to every one by 5000
```

```
UPDATE public.employees  
SET salary=salary+5000;
```

```
UPDATE public.employees  
SET phone_number='590.423.1234', salary=21000  
WHERE employee_id=105;
```

DELETE

DELETE: To remove records from table, by using DELETE we can

1. Delete one or more records based on condition
2. Delete all records

Syntax: DELETE FROM table_name;

DELETE FROM table_name WHERE condition;

INTERVIEW QUESTION

Important interview question:

What is the difference between Delete and Truncate?

Ans:

We can delete one or more or all records from a table by using Delete statement based on conditions and we can rollback the deleted data if we need. Delete is a DML statement. Truncate will delete all the records and the deleted data can't be rolled back. It is a DDL statement.

TCL (TRANSACTION CONTROL LANGUAGE)

TCL: A transaction is set of SQL statements executed on the data stored in Database. TCL commands are used to manage those transactions in the database and mainly to control the transactions made by DML commands like INSERT, UPDATE and DELETE.

COMMIT
ROLLBACK
SAVEPOINT

COMMIT permanently saves changes made in the database.

After commit:

- changes cannot be undone normally.

ROLLBACK cancels changes and restores previous state.

Used when:

- error occurs
- transaction fails

COMMIT

Saves changes permanently

Transaction successful

Like Save

ROLLBACK

Cancels changes

Transaction failed

Like Undo

SAVEPOINT is used to create a temporary point inside a transaction.

It allows partial rollback instead of rolling back the entire transaction.

SAVEPOINT = Checkpoint inside transaction
You can return back to that point if an error occurs.

Normally:

- **ROLLBACK** cancels entire transaction.

But sometimes we want:

- undo only a part of transaction.

So we use SAVEPOINT.

COMMIT: It is used to permanently save any transaction into the database.

Syntax: COMMIT;

ROLLBACK: It is used to rollback the transactions made after last commit.

Syntax: ROLLBACK;

SAVEPOINT: It is used to save transactions up to a point and we can rollback the transactions made after that point.

SAVE TRANSACTION SP1;

transactions....

ROLLBACK TRANSACTION SP1;

DCL (DATA CONTROL LANGUAGE)

DCL: DCL commands are used to handle access privileges for users and roles on database objects to. Only database administrator's & owner's of the database object can provide and remove the privileges.

GRANT: It is used to provide permissions on database objects to users and roles.

Syntax: GRANT privilege_name ON db_object TO user/role

REVOKE: It is used to remove permissions on database objects to users and roles.

Syntax: REVOKE privilege_name ON db_object FROM user/role

Privileges: Privileges let the user know what are all operations that user can perform on that object.

Table/View level privileges: SELECT, INSERT, UPDATE, DELETE and ALL

Schema/Database level privileges: CREATE, ALTER and DROP

DQL (DATA QUERY LANGUAGE)

DQL:

- Select is the only DQL command we use to retrieve data from Database.
- Select is the very important command that we use to fetch data from Tables and Views.
- Select command plays key role in Data analysis.
- Syntax: `SELECT */column_list FROM table/view_name;`

Let's see what are the things we can do with SELECT..

Select:

- Select command is used to retrieve data from Tables and Views in Database.
- Syntax: `SELECT */column_list FROM table/view_name;`

With Select Statement we can

1. Fetch all columns(*) or only required columns from a table
2. Fetch all records or subset of records using where filter condition
3. Evaluate expressions
4. Call pre-defined functions and user defined functions
5. Calculate aggregations and summaries
6. Order data in ascending or descending order

```
-- Calculate yearly salary for all employees
SELECT employee_id, first_name, salary as mon_sal, salary*12 as year_sal
FROM hrdata.employees;
```

```
-- Perform arithmetic operations
SELECT 20+48, 278*349, -146-24;
```

```
-- Call Predefined functions
SELECT sqrt(625), power(4,3), mod(92,10);
```

```
-- Other Databases try with DUAL or DUMMY
SELECT 20+48, 278*349, -146-24, sqrt(625), power(4,3), mod(92,10) FROM dual;
(or)
SELECT 20+48, 278*349, -146-24, sqrt(625), power(4,3), mod(92,10) FROM dummy;
```

results  Chart

20+48	278*349	-146-24
68	97,022	-170

WHERE CLAUSE

Where:

- We use WHERE clause to filter data from tables/views.
- We can mention one or more conditions in WHERE clause based on our requirements.
- The SELECT statement displays all the records satisfying the conditions mentioned in WHERE.

Syntax: `SELECT */column_list FROM table/view_name WHERE condition(s);`

Note: SQL keywords are case in-sensitive, we can use select or SELECT, WHERE or where.
But the data we are operating is case sensitive, so 'a' is different from 'A' in data.

```
-- Display whose yearly income is >= 200000
SELECT employee_id, first_name, salary as mon_sal, salary*12 as year_sal
FROM hrdata.employees WHERE salary*12 >= 200000;

-- List of employees who joined on or after 2000-01-01
SELECT employee_id, first_name, hire_date FROM hrdata.employees
WHERE hire_date >= TO_DATE('2000-01-01', 'YYYY-MM-DD');

-- List of employees whose salary is >= 5000 from department 50 or whose salary >=6000 from dept 80
SELECT employee_id, dept_id, first_name, salary FROM hrdata.employees
WHERE (dept_id=50 AND salary >= 5000) OR (dept_id=80 AND salary >= 6000);
```

ORDER BY CLAUSE

Order by:

- To order the data in ascending order or descending order.
- Default is ascending order, have to specify 'desc' for descending order.
- Can be applied on any data type(Numeric, Strings, Date)
- Can be applied on multiple columns

Syntax:

```
SELECT */column_list FROM table/view_name ORDER BY Col2;
SELECT */column_list FROM table/view_name ORDER BY Col3 desc;
SELECT */column_list FROM table/view_name ORDER BY Col1 desc, col2;
```

STRING MATCHING

Like - To match strings

Not like - Opposite match of strings

Wildcard operators: _ and %

'_' - To match exactly single character

'%' - To match Zero or more characters

```
-- List all employees whose first name starts with letter 'A'
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE first_name LIKE 'A%';
```

-- SQL Data is case sensitive, upper case letter won't match with lower case letters.

```
-- List all employees whose first name doesn't starts with letter 'A'
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE first_name NOT LIKE 'A%';
```

```
-- List all employees where 2nd character of first_name is 'a'
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE first_name LIKE '_a%';
```

```
-- List all employees whose first_name ends with 'a'
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE first_name LIKE '%a';
```

Activate Windows
Go to Settings to activate Windows

```
-- List all employees whose first_name starts with 'A' and ends with 'a'
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE first_name LIKE 'Aa%';
```

```
-- List all employees whose first name and last name starts with letter 'A'
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE first_name LIKE 'A%' AND last_name LIKE 'A%';
```

Activate Windows
Go to Settings to activate Windows

```
-- List all employees whose first name contains a space in it
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE first_name LIKE '% %';
```

```
-- List all employees whose last name has 1 or 2 characters
SELECT employee_id, first_name, last_name FROM hrdata.employees
WHERE last_name LIKE '_' or last_name LIKE '__';
```

PREDEFINED FUNCTIONS

SQL has so many predefined functions..

1. Numeric Functions
2. String Functions
3. Date and Time Functions
4. Conversion Functions
5. Aggregate Functions
6. Null handling Functions
7. Window Functions

NUMERIC FUNCTIONS

ABS - Returns the absolute(positive) value of a numeric expression.

MOD - Equivalent to the % which returns remainder.

SQRT - Returns the square-root of a non-negative number or expression

SQUARE - Returns the square of a numeric expression.

POWER(x,y) – Returns x to the power of y value

CEIL – Returns the nearest equal or larger integer

FLOOR - Returns the nearest equal or smaller integer

ROUND – Returns the nearest integer based on the decimal part

SIGN – Returns sign of its argument(-1 for negative, 1 for positive, 0 for 0)

LOG - Returns the logarithm of a number or expression

LN - Returns the natural logarithm of a number or expression

EXP(x) – Returns Euler number (e – 2. 718282) to the power of x

FACTORIAL – Returns factorial of a number or expression

```
SELECT abs(1.25), abs(-4.71), abs(-981), abs(0);  
SELECT mod(25,4), mod(-39,5), mod(1234,7);  
SELECT sqrt(225), square(12), power(5,4);  
SELECT ceil(2.3), ceil(-4.8), ceil(0);  
SELECT floor(2.3), floor(-4.8), floor(0);  
SELECT round(2.3), round(2.8), round(2.5), round(-4.8);
```

results Chart

MOD(25,4)	MOD(-39,5)	...	MOD(1234,7)
1	-4		2

```
-- Fetch all employee details with even number employee id  
SELECT * FROM hrdata.employees WHERE mod(employee_id,2)=1;
```

```
-- Divide the input data into 4 sets
SELECT * FROM hrdata.employees WHERE mod(employee_id,4)=0;
SELECT * FROM hrdata.employees WHERE mod(employee_id,4)=1;
SELECT * FROM hrdata.employees WHERE mod(employee_id,4)=2;
SELECT * FROM hrdata.employees WHERE mod(employee_id,4)=3;
```

Each set is processed one time separately

If dividing into 3 sets then,

where $\text{mod}(\text{employee_id}, 3) = 0$

where $\text{mod}(\text{employee_id}, 3) = 0$

where $\text{mod}(\text{employee_id}, 3) = 0$

STRING FUNCTIONS

LENGTH – To calculate length of given string

CONCAT – To combine two or more strings

TRIM – To remove spaces on both left and right sides of the given string

LTRIM – To remove spaces or special characters on the left side of the given string

RTRIM – To remove spaces or special characters on the right side of the given string

LPAD – To add spaces or characters on the left side of the given string

RPAD – To add spaces or characters on the right side of the given string

SUBSTR(SUBSTRING) – To get a part of the string

REVERSE – Returns reverse of given string

LOWER – Converts given string to lower case

UPPER – Converts given string to UPPER case

INITCAP – Converts first letter of each word in the given string to UPPER case

REPLACE – To replace one or more characters/words with other words

```
-- trim leading and trailing special characters from a string
SELECT trim('#123#675#912##', '#');
```

```
-- remove leading 0s from the column|
SELECT ltrim('000235', 0);
```

```
-- load and rpad
```

```
SELECT lpad(employee_id, 5, '0'), lpad(first_name, 10, '#'), rpad(last_name, 10, '*') FROM hrdata.employees;
```

```
SELECT load(first_name, 10, '*#&') FROM hrdata.employees;
```

	LPAD(FIRST_NAME, 10, '**&') ...
1	**&*Ramana
2	**&*&Devi
3	**&*Nagesh
4	**&*#Ranga
5	**#Sivarama
6	**&*Chandana

```
-- substr(substring)
SELECT substr('I am learning SQL', 1, 6);
SELECT substr('I am learning SQL', 3, 8);
```

	SUBSTR('I AM LEARNING SQL', 1, 6)	...
1	I am l	

	SUBSTR('I AM LEARNING SQL', 3, 8)	...
1	am learn	

Here indexing starts form 1 only not form 0

```
-- fetch last 5 characters from Last name
SELECT last_name, substr(last_name, length(last_name)-5+1, 5) as fname FROM hrdata.employees;
```

```
-- fetch last 5 characters from Last name
SELECT last_name, substr(last_name, length(last_name)-5+1, 5) as lname FROM hrdata.employees;
SELECT last_name, CASE WHEN length(last_name) < 5 THEN last_name ELSE substr(last_name, length(last_name)-5+1, 5) END as lname FROM hrdata.employees;
```

DATE AND TIME FUNCTIONS

CURRENT_DATE() – Returns today's date in server time zone
CURRENT_TIME() – Returns current time in server time zone
CURRENT_TIMESTAMP() - Returns current timestamp in server time zone
YEAR(date or ts) – Returns Year from given date or timestamp
MONTH(date or ts) - Returns Month from given date or timestamp (1 to 12)
DAY(date or ts) – Returns Day of the month from given date or timestamp (1-31)
WEEK(date or ts) – Returns Week number(1 to 53) of the year from given date or ts
TO_DATE(expr,format) – Converts given expr of certain format to Date
TO_TIMESTAMP(expr,format) – Converts given expr of certain format to Timestamp
DATEDIFF – To calculate difference between 2 dates in years, months, days, hours etc..
TIMESTAMPDIFF – To calculate difference between 2 timestamps in years, months, days, hours..
DATEADD – To add years, months, days, hours etc. to a Date
TIMESTAMPADD – To add years, months, days, hours etc. to a Timestamp
MONTHS_BETWEEN – To calculate the months between two dates or timestamps.

Other Dates:
 SYSDATE()
 DATETIME
 LONG_DATE
 NOW()

Snowflake server is in loss angeles

```
SELECT to_date('2023-05-12', 'YYYY-MM-DD');
SELECT to_date('2023/05/12', 'YYYY/MM/DD');
SELECT to_date('12/May/2023', 'DD/Mon/YYYY');
SELECT to_date('5/12/23', 'MM/DD/YY');
SELECT to_date('23-12-5', 'YY-DD-MM');
SELECT to_date('20230512', 'YYYYMMDD');
```

```
-- to_timestamp
SELECT to_timestamp('2023-05-13 21:32:53');
```

	TO_DATE('2023-05-12', 'YYYY-MM-DD')	TO_DATE('2023/05/12', 'YYYY/MM/DD')	TO_DATE('12/MAY/2023', 'DD/MON/YYYY')	TO_DATE('2023-05-12', 'YYYY-MM-DD')
1	2023-05-12	2023-05-12	2023-05-12	2023-05-12

Everytime the output format is like this.

```

-- TIMESTAMPDIFF
SELECT current_timestamp, timestampdiff(year, '2001-01-01 00:00:00', current_timestamp) diff;
SELECT current_timestamp, timestampdiff(minute, '2023-06-30 00:00:00', current_timestamp) diff;
SELECT current_timestamp, timestampdiff(second, '2023-06-30 09:50:00', current_timestamp) diff;

-- DATEADD(add_what, howmany, date)
SELECT current_date, dateadd(day, 15, current_date);
SELECT current_date, dateadd(hour, 48, current_date);
SELECT current_date, dateadd(hour, 23, current_date);

--TIMESTAMPADD
SELECT current_timestamp, dateadd(day, 15, current_timestamp);
SELECT current_timestamp, dateadd(hour, 48, current_timestamp);
SELECT current_timestamp, dateadd(hour, 23, current_timestamp);

-- MONTHS_BETWEEN(date1, date2)
SELECT round(months_between(current_date, '2020-01-01'::DATE));

-- Calculate the experience of employees in years and months
SELECT employee_id, hire_date,
floor(months_between(current_date, hire_date)/12) || ' YEARS ' || mod(floor(months_between(current_date, hire_date)),12) || ' MONTHS' as
experience
FROM hrdata.employees;

SELECT employee_id, start_date, end_date,
floor(months_between(end_date, start_date)/12) || ' YEARS ' || mod(floor(months_between(end_date, start_date)),12) || ' MONTHS' as
experience
FROM hrdata.job_history;

```

CONVERSION FUNCTIONS

TO_CHAR – To convert a Decimal or Date expression to Char type

TO_DECIMAL or TO_NUMBER – To Convert a string with only numeric characters to Decimal, If string has non-numeric characters, conversion will fail.

TO_DATE(expr, format) – Converts given expr of certain format to Date

TO_TIMESTAMP(expr, format) – Converts given expr of certain format to Timestamp

CAST – To convert any datatype to other datatype.

```

-- TO_CHAR
SELECT to_char(12.34);
SELECT to_char(current_date);
SELECT to_char(current_timestamp);
SELECT to_char('5/17/2023');
SELECT to_char('5/17/2023'::DATE);

-- TO_DECIMAL -- TO_NUMBER -- Both are same
SELECT to_decimal('000056');
SELECT to_decimal('12.34');
SELECT to_decimal('12.34',10,3);
SELECT to_decimal('678a9');

```

Last query lead to error as that is non numeric string

```
-- to_date - to convert any format of input dates to one date-format
SELECT to_date('2023-05-12', 'YYYY-MM-DD');
SELECT to_date('12/May/2023', 'DD/Mon/YYYY');
SELECT to_date('23-12-5', 'YY-DD-MM');
SELECT to_date('20230512', 'YYYYMMDD');
```

```
-- CAST -- CAST(src_expr of one type, tgt_type)
SELECT cast('4.6532' AS NUMBER(8,2));
SELECT cast('4.6572' AS DECIMAL(8,2));
SELECT cast('4.6572' AS INTEGER);
SELECT cast('00123' AS INT);
SELECT cast(current_date AS TIMESTAMP);
SELECT cast(current_TIMESTAMP AS DATE);
SELECT cast('2023-05-17 12:23:56' as TIMESTAMP);
SELECT cast('28-Apr-2023' as DATE);
```

AGGREGATE FUNCTIONS

- Aggregate functions are used to calculate summaries after grouping the data.
- Summaries like Count, Minimum, Maximum, Sum, Average, Mode, Median.
- Aggregate function performs calculation on multiple values of a column and returns a single value.
- If we want to use Aggregate functions over some groups of data, we have to group the data first based on some column(s) or expression(s).
- We use GROUP BY clause to group the data.

Syntax: SELECT *column/expr*, *aggr_fun* FROM *table_name* GROUP BY *column/expr*;

Having: Having clause is used to filter out the data from groups based on conditions, it is used after GROUP BY clause.

Syntax: SELECT *column/expr*, *aggr_fun* FROM *table_name* GROUP BY *column/expr* HAVING *conditions*;

AGGREGATE FUNCTIONS

COUNT – To get the number of records from a table or view or

SUM – To get the sum of all values of a column

AVG – To get average of all values of a column

MAX – To get the biggest values of a column

MIN – To get smallest value of a column

MODE – To get the most frequent value of values of a column

MEDIAN – To get middle value of column

Note: Except COUNT(*) none of the other aggregate functions consider nulls.

INTERVIEW QUESTIONS

Important Interview Questions:

1. What is the difference between WHERE and HAVING?

Ans: Where is used to filter out the data from a table or view based on conditions, but Having is used to filter out the data from groups of records based on specified conditions, Having is used along with GROUP BY clause.

2. Can you use both WHERE and HAVING in same SQL statement?

Ans: Yes, we can use

3. What is the order of execution of WHERE and HAVING in the query?

Ans: SELECT ... FROM WHERE GROUP BY HAVING

```
-- Simple Aggregate Functions
```

```
SELECT COUNT(*) FROM hrdata.employees; -- 107
```

```
SELECT SUM(salary), AVG(salary) FROM hrdata.employees;
```

```
SELECT ROUND(SUM(salary)), cast(AVG(salary) as NUMBER(10,2)) as AVG_SAL FROM hrdata.employees;
```

```
SELECT MIN(salary), MAX(salary) FROM hrdata.employees;
```

```
SELECT MODE(salary), MEDIAN(salary) FROM hrdata.employees;
```

```
select salary, count(1) from hrdata.employees group by salary;
```

```
-- GROUP BY
-- Dept wise employee count
SELECT dept_id, COUNT(*) FROM hrdata.employees GROUP BY dept_id;
```

	↑ DEPT_ID	COUNT(*)
6	60	5
7	70	1
8	80	33
9	90	3
10	100	6
11	110	2
12	null	3

Here as we have used count(*), it gives (considers) null values also.

```
-- Dept wise Sum and Average Salaries
SELECT dept_id, SUM(salary), cast(AVG(salary) as NUMBER(10,2)) as AVG_SAL FROM hrdata.employees GROUP BY dept_id;

-- Dept wise highest salary
SELECT dept_id, MAX(salary) FROM hrdata.employees GROUP BY dept_id;
```

```
SELECT YEAR(hire_date), count(*) FROM hrdata.employees GROUP BY YEAR(hire_date);
```

```
-- Get count of employees hired in each year Dept wise
```

results ~ Chart

YEAR(HIRE_DATE)	COUNT(*)
1,991	1
1,997	28
1,998	23
1,999	18
1,995	4

```
-- Get count of employees hired in each year Dept wise
SELECT YEAR(hire_date), dept_id, count(*) FROM hrdata.employees
GROUP BY YEAR(hire_date), dept_id;
```

	YEAR(HIRE_DATE)	DEPT_ID	COUNT(*)
1	1,987	90	1
2	1,989	90	1
3	1,993	90	1
4	1,990	60	1
5	1,991	60	1
6	1,998	60	1
7	1,999	60	1
8	1,994	100	2

No order here so i use order by

```
SELECT YEAR(hire_date), dept_id, count(*) FROM hrdata.employees
GROUP BY YEAR(hire_date), dept_id
ORDER BY YEAR(hire_date), dept_id;
```

YEAR(HIRE_DATE)	DEPT_ID	... COUNT(*)
1,991	60	1
1,993	90	1
1,994	30	1
1,994	40	1
1,994	70	1
1,994	100	2
1,994	110	2

```
SELECT dept_id, YEAR(hire_date), count(*) FROM hrdata.employees
GROUP BY YEAR(hire_date), dept_id
ORDER BY dept_id, YEAR(hire_date);
```

	DEPT_ID	... YEAR(HIRE_DATE)	COUNT(*)
10	50	1,995	3
11	50	1,996	4
12	50	1,997	11
13	50	1,998	13
14	50	1,999	9
15	50	2,000	4

```
-- HAVING
-- Dept wise employee count that has minimum of 10 employees
SELECT dept_id, COUNT(*) FROM hrdata.employees GROUP BY dept_id HAVING COUNT(*) >= 10;
```

	DEPT_ID	... COUNT(*)
1	50	44
2	80	33

```
-- Depts that has average salary of 10000 and more
SELECT dept_id, cast(AVG(salary) as NUMBER(10,2)) AVG_SAL FROM hrdata.employees
GROUP BY dept_id HAVING AVG(salary) >= 10000;
```

I

```
-- WHERE and GROUP BY
-- Dept wise employee count who are earning greater than 100000 yearly
SELECT dept_id, COUNT(*) FROM hrdata.employees
WHERE salary*12 >= 100000 GROUP BY dept_id;
```

```
-- WHERE, GROUP BY and HAVING
-- List of Depts that hired at least 10 employees after Year 1995
SELECT dept_id, COUNT(*) FROM hrdata.employees
WHERE YEAR(hire_date) > 1995
GROUP BY dept_id HAVING COUNT(*) > 10;
```

	DEPT_ID	COUNT(*)
1	50	41
2	80	33

NULL HANDLING FUNCTIONS

What is a Null?

- A column with NULL value means that column has no value.
- A Null value is neither Zero(0) nor a blank space.
- Length of a null value is null where length of a blank value is 0.
- We can define nullability of a column in the table definition, called NOT NULL constraint.
- If we define any column as NOT NULLABLE then it will not accept nulls, if you try to load nulls into that, the query will fail.
- Null never matches with another null.
- We should be very careful with nulls.

NULL HANDLING FUNCTIONS

IFNULL(col1/expr1, col2/expr2) – If col1 is null then returns col2 otherwise returns col1

NVL (col1/expr1, col2/expr2) – If col1 is null then returns col2 otherwise returns col1

NVL2 (col1/expr1, col2/expr2, col3/expr3) – If col1 is not null then returns col2 otherwise returns col3

COALESCE(<expr1> , <expr2> [, ... , <exprN>], default) – Returns first not-null expression

NVL - no value

NVL has 2 parameters

NVL2 has 3 parameters

```
-- Derive full name of employees
SELECT EMPID, FIRST_NAME || ' ' || MIDDLE_NAME || ' ' || LAST_NAME as FULL_NAME
FROM EMP.PUBLIC.EMPL;
```

If there is any single null in any column, then the whole answer will be null. So that's why we use IFNULL, NVL, NVL2

```
-- IFNULL
SELECT EMPID, IFNULL(FIRST_NAME, '') || ' ' || IFNULL(MIDDLE_NAME, '') || ' ' || IFNULL(LAST_NAME, '') as FULL_NAME
FROM EMP.PUBLIC.EMPL;
```

```
-- NVL
SELECT EMPID, AGE, SALARY FROM EMP.PUBLIC.EMPL;
SELECT EMPID, AGE, NVL(SALARY, 0) FROM EMP.PUBLIC.EMPL;
```

If there is a null value, then it won't consider that record for the sum or avg or anything except count. So that's why I made null into 0

```
-- Get First name of employees, if not get last name, if not get middle name, if no name available load as 'NA'
SELECT EMPID, NVL(FIRST_NAME, NVL(LAST_NAME, NVL(MIDDLE_NAME, 'NA'))) as NAME, AGE, NVL(SALARY, 0)
FROM EMP.PUBLIC.EMPL;
```

```
SELECT EMPID, COALESCE(FIRST_NAME, LAST_NAME, MIDDLE_NAME, 'NA') as NAME, AGE, NVL(SALARY, 0)
FROM EMP.PUBLIC.EMPL;
```

SQL

All About NULL

EMPID	FIRST_NAME	LAST_NAME	AGE	SALARY		
1	Anitha	Rao	28	60000		
2	Vinay	Kumar	31	70000		102
3	Divya		26	50000	Hyderabad	102
4	Rama	Devi	28		Bangalore	103
5	Naresh	Raju	30	65000		101
6	Uma	Shankar		55000	Chennai	102

Null values and empty and black values are different.

All About NULL

- A column with NULL value means that column has no value.
- A Null value is neither Zero(0) nor a blank space.
- Length of a null value is null where length of a blank value is 0.
- We can define nullability of a column in the table definition, called NOT NULL constraint.
- If we define any column as NOT NULLABLE then it will not accept nulls, if you try to load nulls into that, the query will fail.
- Null never matches with another null.
- We should be very careful with nulls.

EMPID	FIRST_NAME	LAST_NAME	AGE	SALARY	LOCATION	DEPTID
1	Anitha	Rao	28	60000	Hyderabad	101
2	Vinay	Kumar	31	70000		102
3	Divya		26	50000	Hyderabad	102
4	Rama	Devi	28		Bangalore	103
5	Naresh	Raju	30	65000		101
6	Uma	Shankar		55000	Chennai	102

1. Derive full name of employees.
2. Calculate average salary of employees.
3. Find list of employees where location information is not available.
4. Joining tables where join keys have nulls.

1. How to query records with Null values?

By using IS NULL

```
select * from emp where last_name is null;
```

```
select * from emp where salary is null;
```

```
select * from emp where location is null;
```

2. Length of Null:

Length of a null value is null where length of a blank value is 0

```
select location, length(location) from emp;
```

```
select first_name, last_name, length(last_name) from emp;
```

```
select first_name, last_name, length(salary) from emp;
```

3. Concatenation with nulls:

We can't concatenate strings and nulls, if we do entire result will become null.

```
select first_name, last_name, first_name || last_name as full_name from EMP;
```

```
select 'RAM' || null as name;
```

```
select 'RAM' || '' as name;
```

4. IFNULL:

How to handle nulls in above case, Answer is with IFNULL.

```
select first_name, last_name, first_name || ifnull(last_name, '') as full_name from EMP;
```

```
select first_name, IFNULL(last_name, 'NA') from emp;
```

5. Aggregate functions with Nulls

→ Null is neither maximum nor minimum

```
select max(salary) from emp;
```

```
select min(salary) from emp;
```

```
select min(ifnull(salary,0)) from emp;
```

→ Count() function will not consider null values.

```
select count(*) from emp; -- 6
```

```
select count(empid) from emp; -- 6
```

```
select count(last_name) from emp; -- 5
```

```
select count(salary) from emp; -- 5
```

```
select count(location) from emp; -- 6
```

5. Aggregate functions with Nulls

→ AVG(), MEAN() will not consider nulls

```
select avg(salary) from emp;
```

```
select avg(ifnull(salary,0)) as avg_sal from emp;
```

```
select avg(case when salary is null then 0 else salary end) as avg_sal from emp;
```

6. Handling Blank spaces

Find list of employees whose location info is not available

```
select * from emp where location is null;
```

```
select * from emp where location is null or location = '';
```

```
select * from emp where location is null or trim(location) = '';
```

Activat
Go to Set

7. Null never matches with another null

```
select case when null = null then 'True' else 'False' end as null_match;
```

→ Joining tables where join key contain nulls.

Table A

ID
1
2
3
null

Table B

ID
1
2
4
null

```
create table table_A(ID int);  
insert into table_A values(1),(2),(3),(null);
```

```
create table table_B(ID int);  
insert into table_B values(1),(2),(4),(null);
```

```
select A.* from table_A A join table_B B  
on A.ID = B.ID;
```

Cl
Go to Set

```
-- 6. Handling Blank spaces
-- Find list of employees whose location info is not available
select * from PUBLIC.EMPLOY where location is null;
select * from PUBLIC.EMPLOY where location is null or location = '';
select * from PUBLIC.EMPLOY where location is null or trim(location) = '';

=====
--7. Null never matches with another null
select case when null = null then 'True' else 'False' end as null_match;
```

WINDOW FUNCTIONS

A window is a group of related rows. A window can consist of one, or multiple rows.

A window function is any function that operates over a window of rows.

RANK – Returns rank over a group of values, skips the series in case of duplicates

DENSE_RANK – Returns rank over a group of values, doesn't skip the series in case of duplicates

ROW_NUMBER – Returns a unique row number for each row within a group of values

LEAD – To get the next row information

LAG – To get the next row information

FIRST_VALUE – Returns the first value within an ordered group of values.

LAST_VALUE – Returns the last value within an ordered group of values.

NTH_VALUE – Returns the nth value within an ordered group of values.

RANK

Rank:

- Returns the rank of a value within an ordered group of values.
- The rank value starts at 1 and continues up sequentially.
- If two values are the same, they have the same rank but skips the next sequence number. 1
- Used to assign rank numbers, find the top values etc.

Syntax:

RANK() OVER ([partition by col/expr] order by col/expr [asc/desc])

DERIVE THE RANK BASED ON SALARY

EMPID	FIRST_NAME	LAST_NAME	AGE	SALARY
1	Anitha	Rao	28	60000
2	Vinay	Kumar	31	70000
3	Divya	Reddy	25	50000
4	Rama	Devi	28	60000
5	Nareesh	Raju	30	55000
6	Uma	Shankar	26	55000
7	Tilak	Varma	29	60000

```
SELECT EMPID, FIRST_NAME, LAST_NAME, AGE,
SALARY,
RANK() OVER(ORDER BY SALARY DESC) as Rank
FROM EMPLOYEES;
```

EMPID	FIRST_NAME	LAST_NAME	AGE	SALARY	Rank
2	Vinay	Kumar	31	70000	1
1	Anitha	Rao	28	60000	2
4	Rama	Devi	28	60000	2
7	Tilak	Varma	29	60000	2
5	Nareesh	Raju	30	55000	5
6	Uma	Shankar	26	55000	5
3	Divya	Reddy	25	50000	7

Activate V

DENSE_RANK

Desne Rank:

- Returns the rank of a value within a group of values, without gaps in the ranks.
- The rank value starts at 1 and continues up sequentially.
- If two values are the same, they have the same rank, but doesn't skip the next sequence number.
- Used to assign rank numbers, find the top values etc.

Syntax:

DENSE_RANK() OVER ([partition by col/expr] order by col/expr [asc/desc])

DERIVE DENSE RANK BASED ON SALARY

EMPID	FIRST_NAME	LAST_NAME	AGE	SALARY
1	Anitha	Rao	28	60000
2	Vinay	Kumar	31	70000
3	Divya	Reddy	25	50000
4	Rama	Devi	28	60000
5	Nareesh	Raju	30	55000
6	Uma	Shankar	26	55000
7	Tilak	Varma	29	60000

```
SELECT EMPID, FIRST_NAME, LAST_NAME, AGE,
SALARY,
DENSE_RANK() OVER(ORDER BY SALARY DESC) as Rank
FROM EMPLOYEES;
```

EMPID	FIRST_NAME	LAST_NAME	AGE	SALARY	DRank
2	Vinay	Kumar	31	70000	1
1	Anitha	Rao	28	60000	2
4	Rama	Devi	28	60000	2
7	Tilak	Varma	29	60000	2
5	Nareesh	Raju	30	55000	3
6	Uma	Shankar	26	55000	3
3	Divya	Reddy	25	50000	4

Activate V

ROW_NUMBER

Row Number:

- Returns a unique row number for each row within a window partition.
- The row number starts at 1 and continues up sequentially.
- If two values are the same, assigns row number randomly.
- Used to assign a sequence number irrespective of duplicates.

Syntax:

ROW_NUMBER() OVER ([partition by col/expr] order by col/expr [asc/desc])

DERIVE ROW NUMBER BASED ON SALARY

	FIRST_NAME	LAST_NAME	AGE	SALARY
1	Anitha	Rao	28	60000
2	Vinay	Kumar	31	70000
3	Divya	Reddy	25	50000
4	Rama	Devi	28	60000
5	Naresh	Raju	30	55000
6	Uma	Shankar	26	55000
7	Tilak	Varma	29	60000

```
SELECT EMPID, FIRST_NAME, LAST_NAME, AGE,  
SALARY,  
ROW_NUMBER() OVER(ORDER BY SALARY DESC)  
as Rank  
FROM EMPLOYEES;
```

EMPID	FIRST_NAME	LAST_NAME	AGE	SALARY	Rank
2	Vinay	Kumar	31	70000	1
7	Tilak	Varma	29	60000	2
4	Rama	Devi	28	60000	3
1	Anitha	Rao	28	60000	4
5	Naresh	Raju	30	55000	5
6	Uma	Shankar	26	55000	6
3	Divya	Reddy	25	50000	7

Activate

WHICH ONE TO CHOOSE?

It's completely depends on our requirements.

1. Suppose if I want to find winners/toppers based on their score/marks then I have to choose Dense_Rank for sure.
2. If I want to choose particular number of winners, I will go with Rank.
3. If I want to just a sequence number to rows based on some column/s, I will go with Row Number.
4. If we are sure there are no duplicate values, we can choose any one but use Row Number to avoid confusions.

SALARY	RANK	DEN_RNK	... ROW_NUM
24000.00	1	1	1
17000.00	2	2	2
17000.00	2	2	3
14000.00	4	3	4
13500.00	5	4	5
13000.00	6	5	6
12000.00	7	6	7
12000.00	7	6	8
12000.00	7	6	9

In row number, for the same salary it is assigning the different numbers i.e 2,3. I think it is based on the first insertion record or randomly assigning.

```
-- Dept wise top 3 salaries
SELECT * FROM
( SELECT dept_id, salary, first_name, last_name,
  DENSE_RANK() OVER(PARTITION BY dept_id ORDER BY SALARY DESC) as rank
  FROM hrdata.employees
) WHERE RANK <= 3
ORDER BY dept_id, rank;
```

	DEPT_ID	SALARY	FIRST_NAME	LAST_NAME	... RANK
1	10	4400.00	Lokesh	Kavuri	1
2	20	13000.00	Sirisha	Pnatham	1
3	20	6000.00	Swapna	Kumari	2
4	30	11000.00	Simha	Reddy	1
5	30	3100.00	Venkat	Rana	2
6	30	2900.00	Ramya	Sagar	3
7	40	6500.00	Bhargav	Amalapuram	1
8	50	8200.00	Roja	Kumari	1
9	50	8000.00	Vijaya	Ramu	2

```
-- Use Case 1. Find the names of students with top 3 marks
SELECT NAME, MARKS, RANK FROM
( SELECT *, DENSE_RANK() OVER(ORDER BY MARKS DESC) as RANK
  FROM STUDENTS
) S
WHERE RANK <=3
ORDER BY RANK;
```

```
-- Use Case 2. Find Gender wise top 3 students
SELECT GENDER, RANK, NAME, MARKS FROM
(SELECT *, DENSE_RANK() OVER(PARTITION BY GENDER ORDER BY MARKS DESC) as RANK
FROM STUDENTS
) S
WHERE RANK <=3
ORDER BY GENDER, RANK;
```

	GENDER	RANK	NAME	MARKS
1	F	1	Lakshmi	95
2	F	2	Satya	91
3	F	2	Sridevi	91
4	F	3	Seetha	82
5	M	1	Venkat	96
6	M	2	Ganesh	95
7	M	3	Lakshman	92

```
-- Use Case 3. Derive EAMCET rank based on the marks given I
SELECT ID, NAME, DOB, MARKS,
ROW_NUMBER() OVER(ORDER BY MARKS DESC, DOB, NAME) as EAMCET_RANK
FROM STUDENTS;
```

	ID	NAME	DOB	MARKS	EAMCET_RANK
1	5	Venkat	2005-09-22	96	1
2	3	Lakshmi	2005-12-15	95	2
3	15	Ganesh	2006-03-24	95	3
4	7	Lakshman	2005-06-24	92	4
5	13	Sridevi	2005-04-19	91	5
6	22	Satya	2005-08-15	91	6
7	23	Srinu	2005-05-28	90	7

If marks are same, it checks DOB, if DOB also same, it checks name

LAG AND LEAD

LAG: Can fetch the value for a particular column from previous row in the same table/group after sorting in some order, without using a self join.

Syntax: LAG (col/expr, [, <offset> , <default>])
OVER ([PARTITION BY <col/expr>] ORDER BY <col/expr> [{ ASC | DESC }])

LEAD: Can fetch the value for a particular column from subsequent(next) row in the same table/group after sorting in some order, without using a self join.

Syntax: LEAD (col/expr, [, <offset> , <default>])
OVER ([PARTITION BY <col/expr>] ORDER BY <col/expr> [{ ASC | DESC }])

Offset means, which previous year

Default means, if that previous year has no data. Then it takes default value.

FIRST_VALUE, LAST_VALUE AND NTH_VALUE

FIRST_VALUE: Returns the first value within an ordered group of values.

Syntax: FIRST_VALUE(<col/expr>)

OVER ([PARTITION BY <col/expr>] ORDER BY <col/expr> [{ ASC | DESC }]

LAST_VALUE: Returns the last value within an ordered group of values.

Syntax: LAST_VALUE(<col/expr>)

OVER ([PARTITION BY <col/expr>] ORDER BY <col/expr> [{ ASC | DESC }]

NTH_VALUE: Returns the nth value within an ordered group of values.

Syntax: NTH_VALUE(<col/expr> , n)

OVER ([PARTITION BY <col/expr>] ORDER BY <col/expr> [{ ASC | DESC }]

-- Window Functions

-- Set up sample data -- Year wise sales details done by employees

```
CREATE OR REPLACE TABLE public.sales(emp_id INT, year INT, revenue NUMBER(10,2));
```

```
INSERT INTO public.sales VALUES
```

```
(1, 2010, 1000),
```

```
(1, 2011, 1500),
```

```
(1, 2012, 500),
```

```
(1, 2013, 750),
```

```
(2, 2010, 10000), (2, 2011, 12500), (2, 2012, 15000), (2, 2013, 20000),
```

```
(3, 2011, 800), (3, 2012, 1200), (3, 2013, 1100);
```

I

-- LAG and LEAD

-- Difference in sales amounts for all employees with prev year

```
SELECT emp_id, year, revenue, LAG(revenue, 1, 0) OVER (PARTITION BY emp_id ORDER BY year) as rev_prev,
```

```
revenue - rev_prev AS diff_in_rev
```

```
FROM public.sales
```

```
ORDER BY emp_id, year;
```

```
SELECT emp_id, year, revenue, LAG(revenue, 1, 0) OVER (PARTITION BY emp_id ORDER BY year) as rev_prev,
```

```
revenue - LAG(revenue, 1, 0) OVER (PARTITION BY emp_id ORDER BY year) AS diff_in_rev
```

```
FROM public.sales
```

```
ORDER BY emp_id, year;
```

	EMP_ID	...	YEAR	REVENUE	REV_PREV	DIFF_IN_REV
1	1		2,010	1000.00	0.00	1000.00
2	1		2,011	1500.00	1000.00	500.00
3	1		2,012	500.00	1500.00	-1000.00
4	1		2,013	750.00	500.00	250.00
5	2		2,010	10000.00	0.00	10000.00
6	2		2,011	12500.00	10000.00	2500.00
7	2		2,012	15000.00	12500.00	2500.00
8	2		2,013	20000.00	15000.00	5000.00

```
SELECT emp_id, year, revenue, LEAD(revenue, 1, 0) OVER (PARTITION BY emp_id ORDER BY year) as rev_next,
       revenue - LEAD(revenue, 1, 0) OVER (PARTITION BY emp_id ORDER BY year) AS diff_in_rev
FROM public.sales
ORDER BY emp_id, year;
```

(or)

I

```
SELECT emp_id, year, revenue, LEAD(revenue, 1, 0) OVER (PARTITION BY emp_id ORDER BY year) as rev_next,
       revenue - rev_next AS diff_in_rev
FROM public.sales
ORDER BY emp_id, year;
```

	EMP_ID	YEAR	REVENUE	REV_PREV	DIFF_IN_REV
1	1	2,010	1000.00	0.00	1000.00
2	1	2,011	1500.00	0.00	1500.00
3	1	2,012	500.00	1000.00	-500.00
4	1	2,013	750.00	1500.00	-750.00
5	2	2,010	10000.00	0.00	10000.00
6	2	2,011	12500.00	0.00	12500.00
7	2	2,012	15000.00	10000.00	5000.00
8	2	2,013	20000.00	12500.00	7500.00
9	3	2,011	800.00	0.00	800.00

-- FIRST_VALUE, LAST_VALUE and NTH_VALUE

-- Get the top salary and least salary in each row

```
SELECT employee_id, first_name, salary,
       FIRST_VALUE(salary) OVER(ORDER BY salary desc) as top_sal,
       LAST_VALUE(salary) OVER(ORDER BY salary desc) as least_sal
FROM hrdata.employees
order by salary desc;
```

	EMPLOYEE_ID	FIRST_NAME	SALARY	TOP_SAL	LEAST_SAL
1	100	Ramana	24000.00	24000.00	2100.00
2	101	Devi	17000.00	24000.00	2100.00
3	102	Nagesh	17000.00	24000.00	2100.00
4	145	Sumanth	14000.00	24000.00	2100.00
5	146	Pavan	13500.00	24000.00	2100.00
6	201	Sirisha	13000.00	24000.00	2100.00
7	205	Sandeep	12000.00	24000.00	2100.00
8	108	Raja	12000.00	24000.00	2100.00

-- Get dept wise top salary and least salary in each row

```
SELECT dept_id, employee_id, first_name, salary,
       FIRST_VALUE(salary) OVER(PARTITION BY dept_id ORDER BY salary desc) as top_sal_of_dept,
       LAST_VALUE(salary) OVER(PARTITION BY dept_id ORDER BY salary desc) as least_sal_of_dept
FROM hrdata.employees
order by dept_id, salary desc;
```

	DEPT_ID	EMPLOYEE_ID	FIRST_NAME	...	SALARY	TOP_SAL_OF_DEPT	LEAST_SAL_OF_DEPT
1	10	200	Lokesh		4400.00	4400.00	4400.00
2	20	201	Sirisha		13000.00	13000.00	6000.00
3	20	202	Swapna		6000.00	13000.00	6000.00
4	30	114	Simha		11000.00	11000.00	2500.00
5	30	115	Venkat		3100.00	11000.00	2500.00
6	30	116	Ramya		2900.00	11000.00	2500.00
7	30	117	Shiva		2800.00	11000.00	2500.00

```
-- Get dept wise 3rd highest salary in each row
SELECT dept_id, employee_id, first_name, salary,
NTH_VALUE(salary,3) OVER(PARTITION BY dept_id ORDER BY salary desc) as nth_sal_of_dept
FROM hrdata.employees
order by dept_id, salary desc;
```

	DEPT_ID	EMPLOYEE_ID	FIRST_NAME	...	SALARY	NTH_SAL_OF_DEPT
1	10	200	Lokesh		4400.00	null
2	20	201	Sirisha		13000.00	null
3	20	202	Swapna		6000.00	null
4	30	114	Simha		11000.00	2900.00
5	30	115	Venkat		3100.00	2900.00
6	30	116	Ramya		2900.00	2900.00
7	30	117	Shiva		2800.00	2900.00

Null means, there are no that many number of records.

```
--Use Case: Calculate the change percentage of employees hired in each year

SELECT
hire_year
,emp_hired
,LAG(emp_hired,1) OVER(ORDER BY hire_year) as emp_hired_prev_year
,round(((emp_hired - emp_hired_prev_year) * 100 ) / emp_hired_prev_year) as emp_hired_change_per
FROM
( SELECT YEAR(hire_date) as hire_year, COUNT(1) as emp_hired FROM hrdata.employees
  GROUP BY YEAR(hire_date)
) emp;
```

Activat
Go to Ser

Here we did not set default value as 0 . if we set to 0 . then during division it will give error.
So that dont set 0's in such cases.

	HIRE_YEAR	...	EMP_HIRED	EMP_HIRED_PREV_YEAR	EMP_HIRED_CHANGE_PER
1	1,987		2	null	null
2	1,989		1	2	-50
3	1,990		1	1	0
4	1,991		1	1	0
5	1,993		1	1	0
6	1,994		7	1	600
7	1,995		4	7	-43
8	1,996		10	4	150

SUBQUERY

- A Subquery is a query within another SQL query, also called as nested query or inner query.
- Can be used with the SELECT, INSERT, UPDATE, and DELETE statements.
- Can be used in Where clause, From clause and in Joins.
- Subquery must be enclosed in parentheses.
- The subquery generally executes first if it has no co-relation with the main query, so the result of a outer query is dependent on subquery or inner query.

Co-related subquery:

- The Subquery that uses the values from outer query or dependent on the outer query is called Co-related subquery.
- In co-related subquery, the result of the outer query is dependent on inner query and result of the inner query is dependent on outer query, hence called co-related.

Activate 1

IN, EXISTS

IN: To match a column or expression against a list of values, the values can be direct or from result of a subquery.

Syntax:

```
SELECT */column_list FROM table WHERE col1 IN (values..);  
SELECT */column_list FROM table WHERE col1 IN (subquery);
```

EXISTS: Returns True or False based on the subquery result that has condition to evaluate.

Syntax:

```
SELECT */column_list FROM table WHERE EXISTS (SELECT * FROM table WHERE condition);
```

Note: NOT IN is just opposite to IN AND NOT EXISTS is just opposite to EXISTS.

```
-- Subqueries in filter conditions(Where)  
  
-- Find the name of employee who is getting highest salary  
SELECT employee_id, first_name, last_name, salary FROM hrdata.employees  
WHERE salary = (SELECT MAX(salary) FROM hrdata.employees);  
  
-- Find the number of employees in Finance dept  
SELECT COUNT(*) FROM hrdata.employees  
WHERE dept_id =  
(SELECT dept_id FROM hrdata.departments WHERE UPPER(department_name) = 'FINANCE');
```

```

-- Create Tables
CREATE TABLE hrdata.employees
( employee_id NUMBER(6)
, first_name VARCHAR(20)
, last_name VARCHAR(25) NOT NULL
, email VARCHAR(25) NOT NULL
, phone_number VARCHAR(20)
, hire_date DATE NOT NULL
, job_id VARCHAR(10) NOT NULL
, salary NUMBER(8,2)
, commission_pct NUMBER(2,2)
, manager_id NUMBER(6)
, dept_id NUMBER(4)
, PRIMARY KEY (employee_id)
) ;

CREATE TABLE hrdata.departments
( dept_id NUMBER(4)
, department_name VARCHAR(30) NOT NULL
, manager_id NUMBER(6)
, location_id NUMBER(4)
, PRIMARY KEY (dept_id)
) ;

```

It uses 2 tables

```

-- Subqueries with 'IN'
-- Find the total number of employees working in depts 10, 20 and 30
SELECT COUNT(*) FROM hrdata.employees WHERE dept_id in (10,20,30);

-- Find the total number of emp working in depts Admin, Marketing and Purchasing
SELECT COUNT(*) FROM hrdata.employees WHERE dept_id in
( SELECT dept_id FROM hrdata.departments WHERE UPPER(department_name) in ('ADMINISTRATION', 'PURCHASING', 'MARKETING')
);

```

Nested subqueries means query within the query and with in the subquery

```

-- Nested Subqueries
-- Find the number of employees working from Seattle location
SELECT COUNT(*) FROM hrdata.employees
WHERE dept_id in
( SELECT dept_id FROM hrdata.departments
  WHERE location_id =
    ( SELECT location_id FROM hrdata.locations WHERE UPPER(city) = 'SEATTLE')
);

```

```

, commission_pct NUMBER(2,2)
, manager_id NUMBER(6)
, dept_id NUMBER(4)
, PRIMARY KEY (employee_id)
) ;

```

```

CREATE TABLE hrdata.departments
( dept_id NUMBER(4)
, department_name VARCHAR(30) NOT NULL
, manager_id NUMBER(6)
, location_id NUMBER(4)
, PRIMARY KEY (dept_id)
) ;

```

```

CREATE TABLE hrdata.locations
( location_id NUMBER(4)
, street_address VARCHAR(40)
, postal_code VARCHAR(12)
, city VARCHAR(30) NOT NULL
, state_province VARCHAR(25)
, country_id CHAR(2)
, PRIMARY KEY (location_id)
) ;

```

Location_id is common in location table and department table
Dept_id is common in dept and employee table

- so from location table we get location and
- from dept table using location we get dept and
- from employee table using dept we get count of employees

```

-- Subqueries with EXISTS and NOT EXISTS
-- Correlated Subqueries
-- Find the total number of employees who are working in 'IT' and 'Sales' depts
SELECT COUNT(*) FROM hrdata.employees e
WHERE EXISTS
( SELECT * FROM hrdata.departments d
  WHERE e.dept_id = d.dept_id and d.department_name in ('IT','Sales')
); -- 39

-- Find the total number of employees who are not working in 'IT' and 'Sales' depts
SELECT COUNT(*) FROM hrdata.employees e
WHERE NOT EXISTS
( SELECT * FROM hrdata.departments d
  WHERE e.dept_id = d.dept_id and d.department_name in ('IT','Sales')
); -- 68

```

CASE STATEMENT

- The CASE statement goes through the conditions specified and returns the value where the condition is true and stops going through the next conditions, if none of the conditions are true then it returns the value specified in the ELSE clause.
- CASE statement works like IF-ELSE (We can't use IF-ELSE in SQL but we can use in PL-SQL).

Syntax:

```
SELECT  
CASE WHEN condition1 THEN result1  
      WHEN condition2 THEN result2  
      WHEN conditionN THEN resultN  
      ELSE result  
END as col_name  
FROM table_name;
```

Here all the results (1,2,3) all should be same data type either int or string or date etc...

DECODE STATEMENT

- Decode compares the column/expression with search value and returns the result value where it matches and if none of the search values are matching with expression it returns the default value specified in the statement.
- It works similar to IF-ELSE and CASE but the difference in CASE and IF-ELSE we can write any conditions, but in DECODE the conditions are based on the column/expression specified.
- This is useful when we need to compare same col/expr with multiple values.

Syntax:

```
SELECT  
DECODE (col/expr, search_1, result_1 [, search_2, result_2].....[,search_n,result_n] [, default])  
FROM table_name;
```

DIFF BETWEEN CASE AND DECODE

Important interview question:

What is the difference between CASE and DECODE? 

CASE	DECODE
Case is a statement	Decode is system function
CASE works with other relational operators like (>, <,>=, <=) along with equal to (=)	Decode only works with equality(=) checks
Case can be used as parameter in Procedure calls	Decode doesn't work in procedure calls
Case statement can be used in PL SQL blocks	Decode works in PL SQL programs but only in select statements
Case expects data type consistency of returning values	Decode does not expect

```
-- Using Case statement to Derive full form of Gender
SELECT empid, empname, gender as gender_code,
CASE WHEN gender = 'F' THEN 'Female'
      WHEN gender = 'M' THEN 'Male'
      WHEN gender = 'T' THEN 'Transgender'
      WHEN gender = 'D' or gender is null THEN 'Don\'t want to disclose'
      ELSE 'Others' END as gender
FROM public.empl;
```

```
-- Using Decode statement to Derive full form of Gender
SELECT empid, empname, gender as gender_code,
DECODE(gender, 'M', 'Male', 'F', 'Female', 'T', 'Transgender',
        'D', 'Don\'t want to disclose', null, 'Don\'t want to disclose', 'Others') as gender
FROM public.empl;
```

Activate
Go to Settings

```
-- Use Case: Derive the tax amount of employees based on their yearly earnings
SELECT
I
employee_id, first_name, last_name, salary,
CASE WHEN salary*12 <= 100000 THEN 0
      WHEN salary*12 between 100001 AND 200000 THEN cast( (salary*10)/100 as NUMBER(10,2))
      WHEN salary*12 between 200001 AND 300000 THEN cast( (salary*20)/100 as NUMBER(10,2))
      ELSE cast( (salary*30)/100 as NUMBER(10,2))
      END as tax_amount
FROM hrdata.employees
ORDER BY salary DESC;
```

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	TAX_AMOUNT
26	152	Preethi	Ganta	9000.00	900.00
27	109	Mohan	Chandra	9000.00	900.00
28	175	Jagannath	Venkata	8800.00	880.00
29	176	Seetha	Kumari	8600.00	860.00
30	177	Tara	Trivedi	8400.00	840.00
31	206	Bhavya	Ramesh	8300.00	0.00
32	121	Roja	Kumari	8200.00	0.00
33	110	Mohan	Venkata	8200.00	0.00
34	120	Vilava	Ramu	8000.00	0.00

CONSTRAINTS IN SQL

Constraints: Constraints are used to specify conditions or rules or restrictions for data in a table. In case of any violation to these constraints, the SQL statements will fail.

Below are the constraints in SQL.

- UNIQUE
- NOT NULL
- DEFAULT
- CHECK
- PRIMARY KEY
- FOREIGN KEY

Column Level Constraints:-

UNIQUE: Unique constraint can be applied at column level, this constraint ensures that all values in that column are different and it won't allow duplicates. It allows null values.

NOT NULL: Not Null can be applied at column level, this constraint won't allow nulls into the column, if you try to load null value then SQL statement gives error.

DEFAULT: Default can be applied at column level, where we have to specify a default value for that column, for any record if we do not insert/load a value to that column then the default value will be stored in that column.

CHECK: Check constraint can be applied at column level, where we have to specify a condition. This allows only the values that are satisfying the condition. If the condition is false then the SQL statement gives error.

Note: We can specify all above constraints at the time of table creation or can add constraints later by using ALTER TABLE statement.

Unique constraint allows multiple nulls

Table Level Constraints:-

CONSTRAINTS IN SQL

PRIMARY KEY:

- Primary Key helps us to identify each row uniquely in a table.
- It is combination of UNIQUE and NOT NULL constraints.
- Primary Key can be applied on a single column or combination of multiple columns.
- This is a table level constraint and can be added at the time of table creation or later by using ALTER TABLE statement.

FOREIGN KEY:

- Foreign Key is used to specify referential integrity between tables.
- That means same column present in two tables, in the first table that column is a Primary Key and in the other table it can be used as foreign key to refer the record from that other table.
- Foreign Key is basically used to provide the links between database tables.
- We use REFERENCES key word to specify foreign key relationships.

```
CREATE TABLE employee (  
  emp_id INT UNIQUE,  
  first_name VARCHAR(30) NOT NULL,  
  last_name VARCHAR(30),  
  salary FLOAT DEFAULT 0,  
  age INT CHECK (age > 18),  
  location VARCHAR(20),  
  dept_id INT REFERENCES dept(dept_id),  
  PRIMARY KEY (emp_id)  
);
```

```
CREATE TABLE dept (  
  dept_id INT UNIQUE,  
  dept_name VARCHAR(30) NOT NULL,  
  manager_id INT,  
  loc_id INT REFERENCES location(loc_id),  
  PRIMARY KEY (dept_id)  
);
```

Important Interview Question:

How many unique constraints and primary key constraints a table can have ?

Ans: A table can have any number of unique keys but only one primary key. That means we can define multiple unique keys on different columns, but can define single primary key at table level.

```
drop table customers;  
drop table orders;  
drop table shippings;
```

```
-----  
CREATE TABLE customers  
(  
    cust_id INTEGER,  
    cust_name VARCHAR(30) NOT NULL,  
    address VARCHAR(50),  
    city_cd VARCHAR(3) references city(city_cd),  
    acc_bal FLOAT check(acc_bal >= 1000),  
    phone VARCHAR(20),  
    PRIMARY KEY (cust_id)  
);  
  
CREATE TABLE orders  
(  
    order_id INTEGER PRIMARY KEY,  
    cust_id INTEGER references customers(cust_id),  
    order_date DATE,  
    total_price FLOAT check(total_price > 0),  
    order_status VARCHAR(20) default 'New'  
);
```

```
CREATE TABLE city  
(  
    city_id INTEGER,  
    city_cd VARCHAR(3) UNIQUE,  
    city_name VARCHAR(20),  
    PRIMARY KEY(city_id)  
);
```

```
PRIMARY KEY (cust_id, cust_name)
```

Multiple columns act as a primary key here

→when we use foreign keys, then first we need to insert the data in base table and then insert in foreign key table or else we will get a foreign key **constraint failed**.

→Check constraint allows nulls or if we are giving a data, it should satisfy the condition.

JOINS IN SQL

JOIN: Join clause is used to combine rows from two or more tables by joining them based on common filed(s) between the tables. We call them as Join Keys. Join Key can be a single column or set of columns.

empid	empname	salary	deptid
1	John	48000	101
2	Charles	55000	102
3	Tina	62000	101

deptid	deptname	loc_cd
101	Sales	BNG
102	Marketing	HYD
103	Finance	CHN

loc_cd	location
BNG	Bangalore
HYD	Hyderabad
CHN	Chennai
MUM	Mumbai

empid	empname	salary	deptname	location
1	John	48000	Sales	Bangalore
2	Charles	55000	Marketing	Hyderabad
3	Tina	62000	Sales	Bangalore

Below are the four types of joins in SQL

1. INNER JOIN
2. LEFT OUTER JOIN
3. RIGHT OUTER JOIN
4. FULL OUTER JOIN

Other than these 4 we have

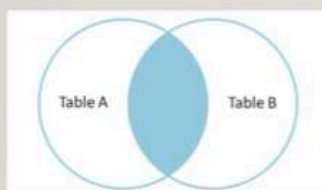
- SELF JOIN
- CARTESIAN JOIN (or) CROSS JOIN

INNER JOIN

INNER JOIN: Returns all matching rows between both the tables based on the Join Key.

Syntax:

```
SELECT A.*, B.*
FROM TABLE_A A INNER JOIN TABLE_B B
ON A.Key = B.Key;
```



Employee Table				Dept Table	
empid	empname	salary	deptid	deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing
3	Tina	62000	103	104	Finance

Employee INNER JOIN Dept On deptid					
empid	empname	salary	E.deptid	D.deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing

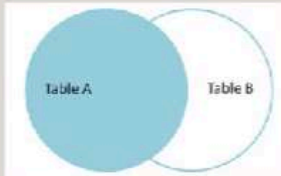
LEFT OUTER JOIN: Returns all rows from Left table and matching rows from Right table. For all non-matching rows from Right table, it will return nulls.

Syntax:

SELECT A.*, B.*

FROM TABLE_A A LEFT OUTER JOIN TABLE_B B

ON A.Key = B.Key;



Employee Table				Dept Table	
empid	empname	salary	deptid	deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing
3	Tina	62000	103	104	Finance

Employee LEFT OUTER JOIN Dept On deptid					
empid	empname	salary	E.deptid	D.deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing
3	Tina	62000	103		

Activate Windows

RIGHT OUTER JOIN

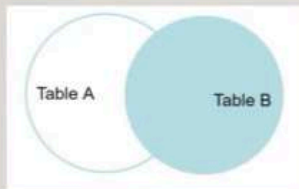
RIGHT OUTER JOIN: Returns all rows from Right table and matching rows from Left table. For all non-matching rows from Left table, it will return nulls.

Syntax:

SELECT A.*, B.*

FROM TABLE_A A RIGHT OUTER JOIN TABLE_B B

ON A.Key = B.Key;



Employee Table				Dept Table	
empid	empname	salary	deptid	deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing
3	Tina	62000	103	104	Finance

Employee RIGHT OUTER JOIN Dept On deptid					
empid	empname	salary	E.deptid	D.deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing
				104	Finance

FULL OUTER JOIN

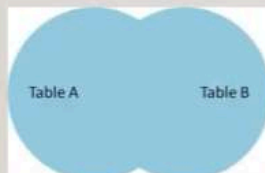
LEFT OUTER JOIN: Returns all rows from both Left and Right tables. For all non-matching rows from both tables, it will return nulls.

Syntax:

SELECT A.*, B.*

FROM TABLE_A A FULL OUTER JOIN TABLE_B B

ON A.Key = B.Key;



Employee Table				Dept Table	
empid	empname	salary	deptid	deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing
3	Tina	62000	103	104	Finance

Employee FULL OUTER JOIN Dept On deptid					
empid	empname	salary	E.deptid	D.deptid	deptname
1	John	48000	101	101	Sales
2	Charles	55000	102	102	Marketing
3	Tina	62000	103		
				104	Finance

Activate Windows

SELF JOIN

SELF JOIN: Sometimes we may need to join a table with same table, we call that join as self join

Syntax:

SELECT A.*, B.*

FROM TABLE_A A INNER JOIN TABLE_A B

ON A.Key = B.Key;

(ON A.managerid = B.empid)

Employee Table			
empid	empname	salary	managerid
1	John	78000	
2	Charles	65000	1
3	Tina	61000	1
4	Bobby	40000	2

			B.empname as managename
empid	empname	salary	
1	John	78000	
2	Charles	65000	John
3	Tina	61000	John
4	Bobby	40000	Charles

CARTESIAN JOIN

CARTESIAN JOIN: Every row of a table will be joined with every row of other table.

There will be no join keys between two tables or no need to specify them.

We get the output row count as multiplication of both tables row count.

Syntax:

SELECT A.*, B.*

FROM TABLE_A A CROSS JOIN TABLE_B B;

(or simply)

SELECT A.*, B.*

FROM TABLE_A , TABLE_B B;

Employee Table			
empid	empname	salary	deptid
1	John	48000	101
2	Charles	55000	102

Dept Table	
deptid	deptname
101	Sales
102	Marketing

Employee CROSS JOIN Dept					
empid	empname	salary	E.deptid	D.deptid	deptname
1	John	48000	101	101	Sales
1	John	48000	102	102	Marketing
2	Charles	55000	101	101	Sales
2	Charles	55000	102	102	Marketing

JOINS IN SQL – POINTS TO REMEMBER

1. We can achieve INNER JOIN with WHERE clause, but INNER JOIN is preferred.
SELECT A.*, B.* from TABLE_A A, TABLE_B B WHERE A.Key=B.Key
2. Be careful where there are nulls in Join keys, null never matches with another null.
3. In real time we mostly use INNER, LEFT OUTER and SELF joins.
4. TABLE_A LEFT OUTER JOIN TABLE_B is same as TABLE_B RIGHT OUTER JOIN TABLE_A.
5. Avoid using Cartesian joins as they are costly that means performance will be very slow.
6. While writing joins we should consider some important points.
 1. What we need in the output exactly
 2. The performance of the query should be good
 3. Query should be in readable format that any one can understand it in future.

In the first point

- 1)if there millions of records it joins the all records and then it applies the WHERE filter, it takes more time
- 2)but when it comes to less number of records(100-220), it works very fast
- 3)so better use inner join

Important Interview question:

Below are two tables with one column in each, what is the output number of records after each type of join?

Table A
1
2
1
3
null

Table B
1
2
2
4
null
null

INNER JOIN –

LEFT JOIN –

RIGHT JOIN –

FULL OUTER JOIN –

CARTESIAN JOIN –

```
-- Inner Join
SELECT emp.employee_id, emp.first_name, emp.last_name, emp.salary, emp.dept_id, dept.dept_id, dept.department_name
FROM hrdata.employees emp
INNER JOIN hrdata.departments dept
ON emp.dept_id = dept.dept_id ;
```

```
-- Left Outer Join
SELECT emp.employee_id, emp.first_name, emp.last_name, emp.salary, emp.dept_id, dept.dept_id, dept.department_name
FROM hrdata.employees emp
LEFT OUTER JOIN hrdata.departments dept
ON emp.dept_id = dept.dept_id ;
```

```
-- Self Join
-- Get Manager Names of all employees
SELECT emp.employee_id,
emp.first_name||' '||emp.last_name as emp_name,
emp.manager_id,
mgr.first_name||' '||mgr.last_name as mgr_name
FROM hrdata.employees emp
JOIN hrdata.employees mgr
ON emp.manager_id = mgr.employee_id;
```

In Inner join, anything is ok WHERE or AND

In left join only WHERE no AND

```
-- Cross or Cartesian Join
SELECT c.*, r.*
FROM hrdata.countries c
CROSS JOIN hrdata.regions r;
```

```
select * from hrdata.countries;
select * from hrdata.regions;
```

[or]

```
SELECT c.*, r.*
FROM hrdata.countries c, hrdata.regions r;
```

```
-- Joins with subqueries
-- Get department name and city, state information of employees
SELECT emp.employee_id, emp.first_name, emp.last_name,
dept_loc.department_name, dept_loc.city, dept_loc.state_province as state
FROM hrdata.employees emp
JOIN
( SELECT dept.dept_id, dept.department_name, loc.city, loc.state_province
  FROM hrdata.departments dept
    JOIN hrdata.locations loc
    ON dept.location_id = loc.location_id
) dept_loc
ON emp.dept_id = dept_loc.dept_id;
```

Important Interview question:

Below are two tables with one column in each, what is the output number of records after each type of join?

Table A
1
2
1
3
null

Table B
1
2
2
4
null
null

INNER JOIN – $1+2+1 = 4$

LEFT JOIN – $4+1+1 = 6$

RIGHT JOIN – $4+1+1+1 = 7$

FULL OUTER JOIN –

$4+1+1+1+1+1 = 9$

CARTESIAN JOIN – $5*6 = 30$

Multiple table joins

```
-- Multiple table joins
-- Use Case: Get all employee details(manager, dept, address, country, region, job, job history)
SELECT
emp.employee_id, emp.first_name||' '||emp.last_name as emp_name, emp.salary,
emp.hire_date, emp.email, emp.phone_number, mgr.first_name||' '||mgr.last_name as mgr_name,
dept.department_name, loc.street_address, loc.postal_code, loc.city, loc.state_province as state,
cnt.country_name as country, reg.region_name as region,
job.job_title, jh.start_date, jh.end_date
_
```

```
FROM hrdata.employees emp

LEFT OUTER JOIN hrdata.employees mgr
ON emp.manager_id = mgr.employee_id

LEFT OUTER JOIN hrdata.departments dept
ON emp.dept_id = dept.dept_id

LEFT OUTER JOIN hrdata.locations loc
ON dept.location_id = loc.location_id

LEFT OUTER JOIN hrdata.countries cnt
ON loc.country_id = cnt.country_id

LEFT OUTER JOIN hrdata.regions reg
ON cnt.region_id = reg.region_id

LEFT OUTER JOIN hrdata.jobs job
ON emp.job_id = job.job_id

LEFT OUTER JOIN hrdata.job_history jh
ON emp.employee_id = jh.employee_id
;
```

SET OPERATORS IN SQL

- Set Operators in SQL are same as Set Operators in Mathematics.
- Used to combine or compare two or more datasets.
- Below are the rules to perform Set operations
 - Number of columns must be same in all Select queries
 - Order of columns must be same in all Select queries
 - Datatypes must be compatible

Below are the 4 set operators in SQL:

1. UNION
2. UNION ALL
3. INTERSECT
4. MINUS

UNION

UNION: Union is used to combine result sets of two or more Select queries. After combining the result sets it eliminates duplicates if any, that means if same record exists in both result sets it shows only one record in the output.

Syntax:

```
SELECT col1, col2, ...col_n FROM TABLE_A
UNION
SELECT col1, col2, ...col_n FROM TABLE_B
UNION
SELECT col1, col2, ...col_n FROM TABLE_C
.....
```

TABLE_A	
empid	name
21	Raja
22	Gopal

TABLE_B	
empid	name
22	Gopal
25	Seetha
28	Kamal

TABLE_A UNION TABLE_B	
empid	name
21	Raja
22	Gopal
25	Seetha
28	Kamal

Joins are used to add columns

Set operations are used to add rown

UNION ALL

UNION ALL: Same as Union but Union all but it doesn't eliminate duplicates.

Syntax:

```
SELECT col1, col2, ...col_n FROM TABLE_A
UNION ALL
SELECT col1, col2, ...col_n FROM TABLE_B
UNION ALL
SELECT col1, col2, ...col_n FROM TABLE_C
.....
```

TABLE_A	
empid	name
21	Raja
22	Gopal

TABLE_B	
empid	name
22	Gopal
25	Seetha
28	Kamal

TABLE_A UNION ALL TABLE_B	
empid	name
21	Raja
22	Gopal
22	Gopal
25	Seetha
28	Kamal

INTERSECT

INTERSECT: It returns only common rows between both the tables.

Syntax:

```
SELECT col1, col2, ...col_n FROM TABLE_A
INTERSECT
SELECT col1, col2, ...col_n FROM TABLE_B
INTERSECT
SELECT col1, col2, ...col_n FROM TABLE_C
.....
```

TABLE_A	
empid	name
21	Raja
22	Gopal

TABLE_B	
empid	name
22	Gopal
25	Seetha
28	Kamal

TABLE_A INTERSECT TABLE_B	
empid	name
22	Gopal

MINUS

MINUS: It returns the records that exists in first result set and do not exists in the next result set. Very useful for data validation between two tables.

Syntax:

```
SELECT col1, col2, ...col_n FROM TABLE_A  
MINUS  
SELECT col1, col2, ...col_n FROM TABLE_B
```

TABLE_A	
empid	name
21	Raja
22	Gopal

TABLE_B	
empid	name
22	Gopal
25	Seetha
28	Kamal

TABLE_A MINUS TABLE_B	
empid	name
21	Raja

TABLE_B MINUS TABLE_A	
empid	name
25	Seetha
28	Kamal

It is used to to check whether complete records are stored or not
Used for data validation

Important Interview question:

What is the difference between UNION and UNION ALL, which gives better performance and which one you choose?

Ans: Union combines two or more result sets and eliminate the duplicate records, but Union All doesn't eliminate duplicates.

Performance wise UNION ALL is better as the task of duplicate elimination is not required here.

So if you are concerned with duplicates better choose UNION and if you are sure that there are no duplicates then choose UNION ALL which gives better performance.

→Union and union all do combine the all records. But union do duplicate elimination while union all dont. So performance wise union all is better.

→In union operation, when column names are different. Then the first table column names will come.

```
SELECT rollno, name, marks, TO_DATE(dob, 'YYYYMMDD') FROM public.students_physics  
UNION  
SELECT stud_no, stud_name, stud_marks, stud_dob FROM public.students_chemistry;
```

```
-- USE CASES  
-- 1. Get subject wise marks of all students  
SELECT sid, sname, 'Maths' as subject, marks FROM public.students_maths  
UNION  
SELECT rollno, name, 'Physics' as subject, marks FROM public.students_physics  
UNION  
SELECT stud_no, stud_name, 'Chemistry' as subject, stud_marks FROM public.students_chemistry  
ORDER BY SID;
```

```

-- 2. Get the list of students who didn't attended all 3 exams together
SELECT * FROM
(
  SELECT sid as student_id, sname as student_name FROM public.students_maths
  UNION
  SELECT rollno, name FROM public.students_physics
  UNION
  SELECT stud_no, stud_name FROM public.students_chemistry
)
MINUS
SELECT * FROM
(
  SELECT sid as student_id, sname as student_name FROM public.students_maths
  INTERSECT
  SELECT rollno, name FROM public.students_physics
  INTERSECT
  SELECT stud_no, stud_name FROM public.students_chemistry
);

-- 3. Get the list of students who attended Physics and Chemistry exams but not Maths
(SELECT rollno as student_id, name as student_name FROM public.students_physics
INTERSECT
SELECT stud_no, stud_name FROM public.students_chemistry
)
MINUS
SELECT sid, sname FROM public.students_maths;

-- 4. Assign the ranks to students who attended all 3 exams based on total marks
SELECT student_id, student_name, total_marks,
DENSE_RANK() OVER(ORDER BY TOTAL_MARKS desc) as RANK
FROM
(
  SELECT student_id, student_name, SUM(marks) as total_marks
  FROM
  (
    SELECT sid as student_id, sname as student_name, marks FROM public.students_maths
    UNION
    SELECT rollno, name, marks FROM public.students_physics
    UNION
    SELECT stud_no, stud_name, stud_marks FROM public.students_chemistry
  ) stud

  WHERE student_id in
  (
    SELECT sid as student_id FROM public.students_maths
    INTERSECT
    SELECT rollno as student_id FROM public.students_physics
    INTERSECT
    SELECT stud_no as student_id FROM public.students_chemistry
  )

  GROUP BY student_id, student_name
) stud_rank
ORDER BY RANK;

```

COMMON TABLE EXPRESSIONS (CTES)

CTES

- A CTE (common table expression) is a named subquery defined in WITH clause.
- We can think of the CTE as a temporary table. The result of the query expression is effectively a table.
- We can define CTE once in the query and can be used at multiple places within the query if needed.
- CTEs can be used in Joins as well.
- We can use CTEs in all DML statements along with Select statement.

Syntax:

```
With  
cte1 as ( ... ),  
cte2 as (...) .... cteN(...)  
SELECT * FROM cte;
```

```
-- Use Case 1: Assign ranks to students who attended all 3 exams based on total marks  
  
WITH all3 as  
( SELECT sid as student_id FROM public.students_maths  
  INTERSECT  
  SELECT rollno as student_id FROM public.students_physics  
  INTERSECT  
  SELECT stud_no as student_id FROM public.students_chemistry  
,  
  
all_stud as  
( SELECT sid as student_id, sname as student_name, 'Maths' as subject, marks  
  FROM public.students_maths  
  UNION  
  SELECT rollno, name, 'Physics' as subject, marks FROM public.students_physics  
  UNION  
  SELECT stud_no, stud_name, 'Chemistry' as subject, stud_marks  
  FROM public.students_chemistry  
,  
  
tot_marks as  
( SELECT student_id, student_name, SUM(marks) as total_marks  
  FROM all_stud  
  WHERE student_id in (SELECT student_id FROM all3)  
  GROUP BY student_id, student_name  
)  
  
SELECT student_id, student_name, total_marks,  
DENSE_RANK() OVER(ORDER BY TOTAL_MARKS desc) as RANK  
FROM tot_marks  
ORDER BY RANK;
```

```
-- Use Case 2: Get the list of employees whose Salary is more than the average salary of their department
|
WITH dept_avg as
( SELECT dept_id, cast(AVG(salary) as NUMBER(10,2)) as dept_avg_sal FROM hrdata.employees
  GROUP BY dept_id
)
SELECT employee_id, first_name, emp.dept_id, salary, da.dept_avg_sal
FROM hrdata.employees emp
JOIN dept_avg da
ON emp.dept_id = da.dept_id
AND emp.salary > da.dept_avg_sal
;
```

PIVOT

Pivot operator converts the rows data of the table into the column data.

This operator supports the built-in aggregate functions AVG, MIN, MAX, COUNT, SUM.

Example:

Input:

(empid, amount, month)

```
(1, 10000, 'JAN'),
(1, 400, 'JAN'),
(2, 4500, 'JAN'),
(2, 35000, 'JAN'),
(1, 5000, 'FEB'),
(1, 3000, 'FEB'),
(2, 200, 'FEB'),
(2, 90500, 'FEB'),
(1, 6000, 'MAR'),
(1, 5000, 'MAR'),
(2, 2500, 'MAR'),
(2, 9500, 'MAR'),
```

Output:

EMPID	'JAN'	'FEB'	'MAR'
1	10400	8000	11000
2	39500	90700	12000

```
select * from monthly_sales
```

```
  pivot (sum(amount) for month in ('JAN', 'FEB', 'MAR')) as p
order by empid;
```

UNPIVOT

Unpivot operator converts the column based data into rows, this is opposite to Pivot.

Example:

Input:

(empid, dept, jan, feb, mar);

```
(1, 'electronics', 100, 200, 300),
```

```
(2, 'clothes', 100, 300, 150),
```

```
(3, 'cars', 200, 400, 100);
```

```
select * from monthly_sales
```

```
  unpivot (sales for month in (jan, feb, mar))
```

```
order by empid;
```

Output:

EMPID	DEPT	MONTH	SALES
1	electronics	JAN	100
1	electronics	FEB	200
1	electronics	MAR	300
2	clothes	JAN	100
2	clothes	FEB	300
2	clothes	MAR	150
3	cars	JAN	200
3	cars	FEB	400
3	cars	MAR	100

→ Evaithe manaki rows or cols loki raveli anukuntunnamo avi bracket lo istham manam

INDEXES IN SQL

INDEXES:

- Index is a special look up table used to speedup the data retrieval from Database tables.
- Indexes are just pointers to original data, they are similar to Index page in the book, to search particular topic in the book (column in the database).
- Indexes improves the performance of SQL queries a lot when we are dealing with millions of rows.
- Index can be created on a single column or combination of multiple columns in a table.

Syntax:

CREATE INDEX *index_name* ON *TABLE_NAME*(*col_name*);

Note: Indexes improves the performance of Select queries but degrades the performance of Insert and Update queries. Sometimes we drop and recreate indexes once the data load is completed.

UNIQUE INDEXES: Unique index on a column doesn't allow duplicate values into that column. It works both as Index and Unique constraint.

Syntax:

CREATE UNIQUE INDEX *index_name* ON *TABLE_NAME*(*col_name*);

COMPOSITE INDEXES: An index created on two or more columns of a table is called Composite Index.

Syntax:

CREATE INDEX *index_name* ON *TABLE_NAME*(*col_1*, *col_2*,);

Dropping an Index:

DROP INDEX *index_name*;

Important Interview Questions:

1. What are indexes and unique indexes?
2. What are the advantages of Indexes?
3. When do you create indexes (or) on which columns you create indexes?

Ans: We create indexes in below scenarios..

- On large tables (avoid indexes on small tables)
- On a column that contains wide range of distinct values
- On column or set of columns those are frequently used in Where clause
- On column or set of columns used frequently as Join keys
- On columns where we don't perform frequent Update operations

```
-- Create Index
CREATE INDEX idx_sid ON students(sid);

-- Create Unique Index
CREATE UNIQUE INDEX uniq_idx_stud ON students(sid, subject);

DROP INDEX idx_sid;
```

VIEWS IN SQL

VIEWS:

- A view is a virtual table that contains rows and columns. It is a schema object that stores SQL statement pointing to original tables and fetches data from these original tables when we query the view.
- A view can be created on multiple tables by joining them.
- A view can be created over another view.
- In some databases we can perform DML operations on a view that has no transformations or joins, but in most of the databases views are read-only and we can't perform DML operations on them.

Creating a view:

```
CREATE VIEW view_name as SELECT col1, col2, ..col_n FROM table_name;
```

Dropping a view:

```
DROP VIEW view_name;
```

Altering or Updating a view:

```
CREATE OR REPLACE VIEW view_name as SELECT col1, col2, ..col_n FROM table_name;
```

Advantages or Uses of Views:

1. Restricting the access to original database tables.
2. Simplifying the SQL queries to end users.
3. We can create multiple views on same table based on the requirement.
4. If we want to hide the original column names, we can rename them in the view definition.
5. We can summarize the data from multiple tables.
6. Views take very little space to store the queries.

Materialized Views:

- A normal view just stores the SQL query not the data or results set.
- A Materialized view is a view that stores the results of the query in that view.
- So when we query the materialized view it returns the data from results stored in that view directly and it will not fetch the data from underlying tables.
- Performance wise materialized views are better than views.

Syntax:

CREATE MATERIALIZED VIEW view_name AS SELECT col1, col2, .. col_n FROM table_name;

Data Refresh in Materialized views:

- So, how the data(stored results) get refreshed when underlying tables gets updated?
- There is no standard way of refreshing data but it is database specific.
- In some databases there is Auto refresh(SQL Server, Snowflake)
- In some databases we have to refresh manually at regular intervals(PostgreSQL, MongoDB)
- Some databases supports both(Oracle, Redshift)

Advantages:

- When we are frequently running same query, better create a materialized view over it.
- When we query large dataset which returns few rows, better use materialized views.

Disadvantages are

- 1)data refresh
- 2)space occupy

What are the differences between views and materialized views?

Views	Materialized Views
It is a virtual tables, just stores the SQL query	It stores the result set of the query
Doesn't occupy storage	Storage space needed to store result set
Query fetches data from underlying tables, so performance may be slow	Performs better than Views as it fetches data from pre computed result sets
No data refresh headache	Data refresh is cost oriented

-- Views with Filters, Joins and Aggregates

-- 1. Create a view with details of employees working in 'IT' department

```
CREATE VIEW hrdata.v_employees_it_dept
AS
SELECT emp.*, dept.department_name
FROM hrdata.employees emp
JOIN hrdata.departments dept
ON emp.dept_id = dept.dept_id
WHERE dept.department_name LIKE 'IT'
;
```

```
SELECT * FROM hrdata.v_employees_it_dept;
```

```

-- 5. Create a view with the number of direct employees working under all managers
CREATE VIEW hrdata.v_direct_reportees_count
AS
SELECT emp.manager_id, mgr.first_name||' '||mgr.last_name as mgr_name, COUNT(*) as reportees_count
FROM hrdata.employees emp
JOIN hrdata.employees mgr
ON emp.manager_id = mgr.employee_id
GROUP BY emp.manager_id, mgr_name
;

```

```

CREATE VIEW hrdata.v_dept_wise_emp_hired
AS
SELECT dept.department_name, count(*) as no_emp_hired
FROM hrdata.employees emp
JOIN hrdata.departments dept
ON emp.dept_id = dept.dept_id
WHERE YEAR(hire_date) = 1997
--WHERE YEAR(hire_date) = YEAR(CURRENT_DATE)
GROUP BY dept.department_name
;

```

```

-- Altering or modifying a view definition
-- Just use CREATE OR REPLACE VIEW syntax
CREATE OR REPLACE VIEW hrdata.v_dept_wise_emp_hired
AS
SELECT dept.department_name, count(*) as no_emp_hired
FROM hrdata.employees emp
JOIN hrdata.departments dept
ON emp.dept_id = dept.dept_id
WHERE YEAR(hire_date) = 1999
--WHERE YEAR(hire_date) = YEAR(CURRENT_DATE)
GROUP BY dept.department_name
;

```

DCL (DATA CONTROL LANGUAGE)

Access controlling can be done by DCL commands in SQL.

DCL: DCL commands are used to handle access privileges for users and roles on database objects. Only database administrator's & owner's of the database object can provide and remove the privileges.

GRANT: It is used to provide permissions on database objects to users and roles.

Syntax: GRANT privilege_name ON db_object TO user|role

REVOKE: It is used to remove permissions on database objects to users and roles.

Syntax: REVOKE privilege_name ON db_object FROM user|role

USERS: User can be a person or batch id who is operating or running SQL queries on database tables.

PRIVILEGES: Privileges let the user know what are all operations that user can perform on that object.

ROLES: A Role is a database object that group together a set of privileges that can be assigned to users.

Ex: Developer, Tester, Analyst, Administrator, EndUser etc.

Table/View level privileges: SELECT, INSERT, UPDATE, DELETE and ALL

Schema/Database level privileges: CREATE, ALTER and DROP

```
CREATE ROLE Administrator;-

CREATE ROLE Developers;

CREATE ROLE Analyst;

CREATE ROLE Business_Users;

CREATE ROLE Sales_Users;

CREATE ROLE Marketing_Users;

CREATE ROLE HR;

-- Creating Users

CREATE USER Akhil PASSWORD = 'abc123'
DEFAULT_ROLE = Administrator
MUST_CHANGE_PASSWORD = TRUE;

GRANT ROLE Administrator to USER Akhil;
```

```

-- Administrator Role

-- Granting usage to Warehouse
GRANT USAGE ON WAREHOUSE COMPUTE_WH TO ROLE Administrator;

-- Granting usage to Database
GRANT USAGE ON DATABASE EMP TO ROLE Administrator;

-- Granting Ownership to Database
GRANT OWNERSHIP ON DATABASE EMP TO ROLE Administrator;

-- Revoke usage on Database
REVOKE USAGE ON DATABASE EMP FROM Administrator;

-- Retry Granting Ownership
GRANT OWNERSHIP ON DATABASE EMP TO ROLE Administrator;

-- Granting Ownership on schemas
GRANT OWNERSHIP ON SCHEMA EMP.HRDATA TO ROLE Administrator;
GRANT OWNERSHIP ON SCHEMA EMP.PUBLIC TO ROLE Administrator;

```

PERFORMANCE TUNING

Performance Tuning:

- Tuning is the process of improving your SQL queries to run faster.
- Also called as Query Optimization.
- To reduce the consumption of resources and to reduce waiting time of data retrieval.

Reasons for slow performance:

- Huge tables
- More number of Joins
- More aggregations
- Inefficient query writing

Activate Windows
Go to Settings to activate Windows.

Tuning Process:

1. First analyze the query why it is taking long time to run.
2. Find which step or which part of the code is taking long time.
(Check Execution plan or Query plan or Query profile to find out this)
3. Apply tuning on that part or rewrite the query in other way.
4. Follow some standard tuning techniques.

SQL TUNING TECHNIQUES (ALL ON-PREMISE DATABASES)

- First and important step is, create proper indexes, make use of function based indexes as well.
- Select only required fields, don't use select * always.
- Avoid using multiple OR condition in Where clause, instead you can use Union or IN based on the conditions.
- Avoid using not equal to operator (<> or !=), Instead use Not In.
- Avoid using Distinct, instead we can use Group By on All columns.
- Use Union All instead of Union if you are sure there are no duplicates.
- Avoid joining tables using Where clause it will perform cross join first and then filters the records, Use Inner Join.
- If you want to see sample data then restrict the data using Top 10 or Limit 10.
- If possible avoid writing co-related subqueries.
- Use CTEs instead of subqueries if you want to use same subquery at multiple places in the query.
- Collecting the missing stats on table will help in Teradata.
- Remove unnecessary ORDER BY clauses.

Activate Windows
Go to Settings to activate

PERFORMANCE IN SNOWFLAKE

There are lot of in-built features in snowflake to improve the performance, so first read and understand the concepts.

1. Data Storage:

The data is stored in micro-partitions after compression, if we have large tables then define cluster keys on the frequently used columns of the table, they will help to scan only required micro-partitions to fetch the data instead of scanning all partitions. Cluster keys behave like indexes in on-premise databases.

2. Metadata:

Snowflake manages meta data like number of rows, distinct values of each column, minimum and maximum of values stored in each column. So when you try to fetch the table count or minimum or maximum values, the query will use metadata instead of scanning entire table.

Activate Windows
Go to Settings to activate

3. Caching:

Snowflake has 2 types results cache and disk cache. The result of a query will be stored in the results cache until 24 hours, so if we run the same query again with-in 24 hours, the query will fetch the data from results cache instead of scanning entire table again unless until the data is not updated. In case of disk cache the results will be stored in the memory and this memory will be effective until we close that session.

4. Vertical scaling and Horizontal scaling:

In Snowflake we have flexibility to increase warehouse(Actual processing unit). We have sizes from X-Small to 6XL and the cost will get doubled when we upgrade to next level. But if you are dealing with very large tables and have multiple joins then increasing the size of warehouse to next level will help the query to run faster. This is called Vertical scaling. Suppose if we have so many concurrent queries running and the queries are going into waiting queues then we can go for multi clustering where we increase the number of clusters which is called as Horizontal scaling.

Activate Windows
Go to Settings to activate

5. Dedicated virtual warehouses:

If we have multiple teams working or accessing same database then better use dedicated virtual warehouse for each team like one for Development team, one for Analyst team, One for End users etc. And always use dedicated warehouse for batch processing.

After following all above things, if your query is still running slow then use the SQL performance tuning techniques discussed in previous slide.

Batch processing means, in some scheduled time sql queries will run and load the data. We will give separate DB to this

And give separate DB's for multiple teams

BEST PRACTICES IN WRITING SQL QUERIES

- Select only required fields.
- Write appropriate comments.
- Use proper alias names.
- Follow standard naming conventions for your table names, view names and field names.
- Format your queries, it should be easily understandable to others.

ADVANCED QUERIES

1. How to Calculate Cumulative Sum?
2. Ho to Eliminate Duplicate pair of records
3. Ho to Eliminate Duplicate pair of records using SPLIT_PART
4. How to add up values into single row using LISTAGG?
5. How to generate Sequence Number for existing records?
6. Working with Regular Expressions